

Part I: Divide and Conquer

Lecture 2: Maximum Contiguous Subarray Problem

Computer Science and Technology
United International College

- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem
- 3 A brute force algorithm
- 4 A data-reuse algorithm
- 5 A divide-and-conquer algorithm
- 6 Analysis of the divide-and-conquer algorithm
- 7 Summary

Introduction to Part I

- Divide and conquer (D&C) is an important algorithm design paradigm.
 - Divide
 - Divide a given problem into two or more subproblems (ideally of approximately equal size)
 - Conquer
 - Solve each subproblem (directly if small enough or **recursively**)
 - Combine
 - Combine the solutions of the subproblems into a global solution
- In Part I, we will illustrate Divide-and-Conquer using several examples:
 - Maximum contiguous subarray
 - Polynomial Multiplication
 - Partition and Selection

- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem**
- 3 A brute force algorithm
- 4 A data-reuse algorithm
- 5 A divide-and-conquer algorithm
- 6 Analysis of the divide-and-conquer algorithm
- 7 Summary

Objective and Reference

Objective: Discuss first example of divide-and-conquer

Reference: Chapter 8 in [Programming Pearls, \(2nd ed\)](#) by Jon Bentley

Maximum Contiguous Subarray Problem

Year	1	2	3	4	5	6	7	8	9
Profit M\$	-3	2	1	-4	5	2	-1	3	-1

Table: ACME CORP¹– PROFIT HISTORY

Between years 1 and 9:

- ACME earned $-3 + 2 + 1 - 4 + 5 + 2 - 1 + 3 - 1 = 4$ M\$

Between years 2 and 6:

- ACME earned $2 + 1 - 4 + 5 + 2 = 6$ M\$

Between years 5 and 8:

- ACME earned $5 + 2 - 1 + 3 = 9$ M\$

Problem: Find the span of years in which ACME earned the **most**

Answer: Year 5-8 , 9 M\$

Definition

Let $R[1 \cdots n]$ be an array of n real numbers. The **value** of **subarray** $R[i \cdots j]$ is

$$V(i, j) = \sum_{x=i}^j R(x)$$

Definition(Maximum Contiguous Subarray Problem)

- **Input:** An array $R[1 \cdots n]$ of n real numbers.
- **Output:** $V(i, j)$ such that $\forall(i', j'), V(i', j') \leq V(i, j)$
- We can modify the problem so that it returns indices (i, j) .

- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem
- 3 A brute force algorithm**
- 4 A data-reuse algorithm
- 5 A divide-and-conquer algorithm
- 6 Analysis of the divide-and-conquer algorithm
- 7 Summary

Brute Force

Calculate the value of $V(i, j)$ for each pair $i \leq j$ and return the maximum value

Algorithm 1 *BruteForceMCS(R)*

Input: An array R of n reals

Output: The value of MCS

```
1:  $VMAX = A[1]$ 
2: for  $i = 1$  to  $N$  do
3:   for  $j = i$  to  $N$  do           // calculate  $V(i, j)$ 
4:      $V = 0$ 
5:     for  $x = i$  to  $j$  do
6:        $V = V + R[x]$ 
7:     end for
8:     if  $V > VMAX$  then  $VMAX = V$ 
9:     end if
10:  end for
11: end for
12: return  $VMAX$ 
```

$\Theta(n^3)$ arithmetic additions

- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem
- 3 A brute force algorithm
- 4 A data-reuse algorithm**
- 5 A divide-and-conquer algorithm
- 6 Analysis of the divide-and-conquer algorithm
- 7 Summary

Reuse Data

Idea:

- don't need to calculate each $V(i, j)$ from scratch
- exploit the fact: $V(i, j) = \sum_{x=i}^j R[x] = V(i, j-1) + A[j]$

Algorithm 2 *ReuseDataMCS*(R)

Input: An array R of n integers

Output: The value of MCS

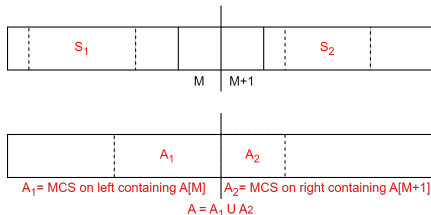
```
1:  $VMAX = R[1]$ 
2: for  $i = 1$  to  $N$  do
3:    $V = 0$ 
4:   for  $j = i$  to  $N$  do           // calculate  $V(i, j)$ 
5:      $V = V + R[j]$ 
6:     if  $V > VMAX$  then  $VMAX = V$ 
7:   end if
8: end for
9: end for
10: return  $VMAX$ 
```

$\Theta(n^2)$ arithmetic additions

- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem
- 3 A brute force algorithm
- 4 A data-reuse algorithm
- 5 A divide-and-conquer algorithm**
- 6 Analysis of the divide-and-conquer algorithm
- 7 Summary

Divide-and-Conquer

Set $M = \lfloor N/2 \rfloor$



The MCS S must be **one** of

- ① S_1 : the MCS in $R[1...M]$
- ② S_2 : the MCS in $R[M+1...N]$
- ③ A : the MCS across the cut.

So,

$$S = \text{the best among } \{S_1, S_2, A\}$$

Example

1	-5	4	2	-7	3	6	-1		2	-4	7	-10	2	6	1	3
---	----	---	---	----	---	---	----	--	---	----	---	-----	---	---	---	---

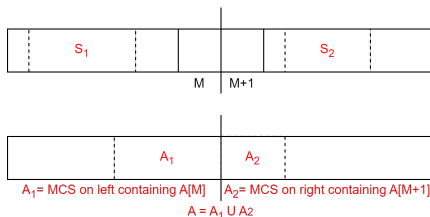
- $S_1 = [3, 6]$ and $S_2 = [2, 6, 1]$

1	-5	4	2	-7	3	6	-1		2	-4	7	-10	2	6	1	3
---	----	---	---	----	---	---	----	--	---	----	---	-----	---	---	---	---

- $A_1 = [3, 6, -1]$ and $A_2 = [2, -4, 7]$
- $A = A_1 \cup A_2 = [3, 6, -1, 2, -4, 7]$
- $Value(S_1) = 9$; $Value(S_2) = 9$; $Value(A) = 13$
- Solution: **A**

MCS across the cut

Let $M = \lfloor N/2 \rfloor$



$$A = A_1 \cup A_2$$

- A_1 : MCS among contiguous subarrays **ending at $R[M]$** (the maximum suffix of $R[1 \cdots M]$)
- A_2 : MCS among contiguous subarrays **starting at $R[M+1]$** (the maximum prefix of $R[M+1 \cdots n]$)

Conquer: Finding the A subarrays



A_1 is in the form $R[i \cdots M]$, $V(k, M) = V(k+1, M) + R[k]$

Algorithm 3 *MaxSuffix*(R, i, M)

Input: $R[i \cdots M]$

Output: The value of A_1

- 1: $MAX = R[M]$
 - 2: $SUM = R[M]$
 - 3: **for** $k = M - 1$ **downto** i **do**
 - 4: $SUM = SUM + R[k]$
 - 5: **if** $SUM > MAX$ **then** $MAX = SUM$
 - 6: **end if**
 - 7: **end for**
 - 8: **return** $Value(A_1) = MAX$
-

Conquer: Finding the A subarrays

There are only $M - i + 1$ sequences of the form

- A_1 can be found in $\mathcal{O}(M - i)$ time

Similarly, A_2 is in the form $R[M + 1 \cdots j]$

- assume the algorithm is called " $MaxPrefix(R, M, j)$ "
- there are only $j - M$ such sequences
- A_2 can be found in $\mathcal{O}(j - M)$ time

$A = A_1 \cup A_2$ can be found in $\mathcal{O}(j - i)$ time

- **linear** to the input size

The Full Divide-and-Conquer Algorithm

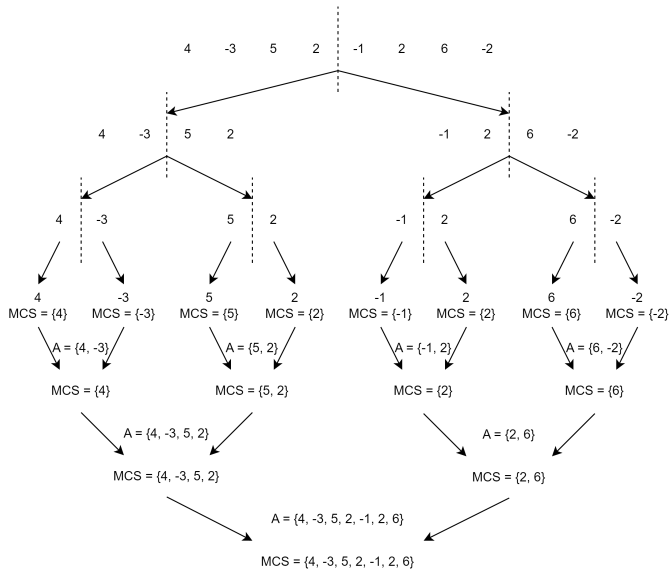
Algorithm 4 $MCS(R, i, j)$

Input: $R[i \cdots j]$ with $i \leq j$

Output: MCS of $R[i \cdots j]$

- 1: **if** $i = j$ **then**
 - 2: **return** $R[i]$
 - 3: **else**
 - 4: $S_1 = MCS(R, i, \lfloor \frac{i+j}{2} \rfloor)$
 - 5: $S_2 = MCS(R, \lfloor \frac{i+j}{2} \rfloor + 1, j)$
 - 6: $A = MaxSuffix(R, i, \lfloor \frac{i+j}{2} \rfloor) + MaxPrefix(R, \lfloor \frac{i+j}{2} \rfloor + 1, j)$
 - 7: **return** $MAX(S_1, S_2, A)$
 - 8: **end if**
-

A Full Example



- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem
- 3 A brute force algorithm
- 4 A data-reuse algorithm
- 5 A divide-and-conquer algorithm
- 6 Analysis of the divide-and-conquer algorithm**
- 7 Summary

- n : problem size ($n = j - i + 1$)
- $T(n)$: time cost for $MCS(R, i, j)$
- the initial call is $MCS(R, 0, n - 1)$

Algorithm 4 $MCS(R, i, j)$

Input: $R[i \cdots j]$ with $i \leq j$

Output: MCS of $R[i \cdots j]$

```
1: if  $i = j$  then
2:   return  $R[i]$  //  $\mathcal{O}(1)$ , base case
3: else
4:    $S_1 = MCS(R, i, \lfloor \frac{i+j}{2} \rfloor)$  //  $T(\frac{n}{2})$ 
5:    $S_2 = MCS(R, \lfloor \frac{i+j}{2} \rfloor + 1, j)$  //  $T(\frac{n}{2})$ 
6:    $A = MaxSuffix(R, i, \lfloor \frac{i+j}{2} \rfloor) + MaxPrefix(R, \lfloor \frac{i+j}{2} \rfloor + 1, j)$ 
   //  $2\mathcal{O}(n)$ 
7:   return  $MAX(S_1, S_2, A)$  //  $\mathcal{O}(1)$ 
8: end if
```

$$T(n) = \begin{cases} \mathcal{O}(1) & (n = 1) \\ 2T(\frac{n}{2}) + \mathcal{O}(n) & (n > 1) \end{cases}$$

Assume that n is a power of 2. Let $h = \log_2 n$, so that $2^h = n$.

$$\begin{aligned} T(n) &\leq 2T\left(\frac{n}{2}\right) + cn \\ &\leq 2\left(2T\left(\frac{n}{2^2}\right) + c\frac{n}{2}\right) + cn \\ &= 2^2 T\left(\frac{n}{2^2}\right) + 2cn \\ &\leq 2^2 \left(2T\left(\frac{n}{2^3}\right) + c\frac{n}{2^2}\right) + 2cn \\ &= 2^3 T\left(\frac{n}{2^3}\right) + 3cn \\ &\leq \dots \\ &= 2^h T\left(\frac{n}{2^h}\right) + hcn \end{aligned}$$

$$T(n) \leq nT(1) + (\log_2 n)cn = \mathcal{O}(n \log_2 n)$$

- 1 Introduction to Part I
- 2 The Maximum Contiguous Subarray Problem
- 3 A brute force algorithm
- 4 A data-reuse algorithm
- 5 A divide-and-conquer algorithm
- 6 Analysis of the divide-and-conquer algorithm
- 7 Summary**

In this lecture we saw 3 different algorithms for solving the maximum contiguous subarray problem

- 1 A $\Theta(n^3)$ **brute force** algorithm
- 2 A $\Theta(n^2)$ algorithm that **reuses data**
- 3 A $\Theta(n \log n)$ **divide-and-conquer** algorithm

Can you solve the problem in $\mathcal{O}(n)$ time?